Quality of Experience and Reliability-Aware Task Offloading and Scheduling for Multi-User Mobile-Edge Computing Systems

Junlong Zhou, *Member, IEEE*, Xiangpeng Hou, *Student Member, IEEE*, Yue Zeng, Peijin Cong, Weiming Jiang, and Song Guo, *Fellow, IEEE*

***Abstract*—Mobile-edge computing (MEC) has received wide attention recently due to its efficacy in alleviating the computation stress of mobile devices (MDs), which is realized by offloading workloads from MD users to nearby edge servers (ESs). Prior work has studied related task offloading and scheduling problems and proposed many approaches. However, none of these approaches considers the reliability issue in MEC systems which may suffer soft errors during task execution as well as bit errors during task offloading simultaneously. Targeting optimization on a multi-user MEC system, in this article we investigate the task offloading and scheduling problem of maximizing system quality of experience (QoE) under a certain reliability requirement. With the consideration of the combinatorial nature of this problem, we propose to decompose the original problem into i) a task-to-ES assignment problem with fixed task offloading decision, for satisfying system reliability constraint, ii) a computing resource allocation problem with fixed task offloading and assignment decisions, for maximizing system QoE, and iii) a task offloading optimization problem to find the best offloading decision that achieves the maximum QoE under the reliability constraint using our task assignment and resource allocation methods. In order to solve these sub-problems, we further design a reliability-aware task-to-ES assignment algorithm, a QoE-optimum resource allocation algorithm, and a binary particle swarm optimization based task offloading algorithm. We perform extensive simulations and testbed experiments to validate the efficacy of the proposed scheme. Simulation and testbed results show that the proposed scheme greatly outperforms four benchmark approaches and it achieves up to 63.2% and 43.1% increase in the average QoE (quantified by offloading utility), respectively.**

***Index Terms*—Mobile-edge computing, quality of experience, reliability, task offloading and scheduling.**

Received 31 August 2024; revised 31 January 2025; accepted 6 March 2025. Date of publication 18 March 2025; date of current version 12 June 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62302221 and Grant 62172224, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20220138 and Grant BK20230913, in part by the Fundamental Research Funds for the Central Universities under Grant 30922010318 and Grant 30924010815, and in part by the Open Research Fund of National Mobile Communications Research Laboratory, Southeast University under Grant 2024D07. *(Corresponding author: Peijin Cong.)*

Junlong Zhou is with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China, and also with the National Mobile Communications Research Laboratory, Southeast University, Nanjing 211189, China (e-mail: jlzhou@njust.edu.cn).

Xiangpeng Hou, Yue Zeng, Peijin Cong, and Weiming Jiang are with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing 210094, China (e-mail: cpj@njust.edu.cn).

Song Guo is with the Hong Kong University of Science and Technology, Hong Kong, China.

Digital Object Identifier 10.1109/TSC.2025.3552338

I. Introduction

With the advances in Internet of Things (IoT) technologies, a variety of mobile applications such as smart phones and autonomous driving are emerging and gradually changing people's lives [2], [3], [4]. These mobile applications typically generate a massive amount of data and thus have a heavy demand for computation capability and energy of mobile devices (MDs) to process massive IoT data. According to the statistics [5], the amount of IoT data is expected to reach 73.1 ZB by 2025. However, most of general MDs cannot meet this demand because of their insufficient computing resources and finite battery energy. A common solution is that mobile applications send their tasks to cloud data centers (CDCs) that have rich computing resources, sufficient energy and huge storage. But CDCs are usually deployed in a remote place far away from MDs, leading to unacceptable transmission delay that would degrade the real-time performance of latency-sensitive applications [6]. As a promising paradigm, mobile edge computing (MEC) [7], [8], [9] is emerging recently to overcome the shortcoming of CDCs. MEC allows MDs to communicate with base stations (BSs) via radio access networks to offload computation-intensive and energy-consuming applications to nearby edge servers (ESs). Compared to the servers in CDCs, ESs of the MEC system are deployed at the network edge which is very close to MDs geometrically, and hence the communication delay is lower. Benefiting from MEC, MDs' computation capability and energy efficiency could be both effectively improved with low latency.

In MEC systems, task offloading and scheduling are two key operations determining which task should be offloaded to which ES for remote execution with what resource, to meet diverse performance requirements so as to guarantee the quality of experience (QoE) of MD users [10], [11], [12]. Task execution latency and device battery energy are two major factors impacting the QoE of MD users [1], [13], [14]. Specifically, the shorter latency and the lower energy consumption, the better user experience. In addition to the QoE of MD users, reliability is also an imperative concern for guaranteeing successful task offloading and execution. In MEC, no matter whether tasks are executed locally on MDs or remotely on ESs, they all possibly suffer soft errors. Besides, the tasks to be offloaded on ESs also probably suffer bit errors during the data transmission. The two types of

1939-1374 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

errors both may adversely impact system function correctness, even leading to serious consequences [15]. Therefore, it's highly necessary to consider QoE and reliability simultaneously when performing task offloading and scheduling for multi-user MEC (MU-MEC) systems.

Facing these challenges, numerous studies [13], [14], [16], [17], [18], [19], [20] have been conducted to promote QoE of users by optimizing execution latency and/or energy consumption of MDs. However, these studies ignore the transmission reliability and execution reliability which are both of vital importance to MEC systems. A few redundancy based approaches [21], [22], [23] could be applied to cope with the execution reliability issues in MEC systems. But they would induce extra time and energy overheads and neglect the transmission reliability of task offloading. This paper focuses on the joint task offloading and scheduling problem with the goal of maximizing the QoE of an MU-MEC system while meeting the reliability requirement. To tackle this problem, we need to decide i) which tasks to be offloaded, ii) the assignment of offloaded tasks to ESs, and iii) the allocation of computing resources to tasks offloaded on ESs. Correspondingly, we solve this problem in three stages. In the first stage, given an initial offloading decision, a reliability-aware task-to-ES assignment method is proposed to judiciously assign the tasks to be offloaded to ESs for increasing system reliability. In the second stage, given task offloading and assignment strategies, we formulate the QoE-maximized resource allocation problem as a convex optimization problem (COP) and analytically address the COP to derive the optimum computing resource allocation solution. In the last stage, we design a heuristic to quickly search the best task offloading decision which can get the global optimum QoE under the reliability constraint using our task assignment and resource allocation approaches. Our main contributions are highlighted as follows.

1) We are the first to take into account the transmission reliability and execution reliability of MU-MEC systems simultaneously. To improve system reliability, we propose a theorem upon the reliability-optimality of task-to-ES assignment and develop a reliability-aware task-to-ES assignment heuristic following the theorem.
2) We formulate and address the computing resource allocation problem for tasks offloaded on ESs to obtain a maximal system QoE (quantified by system utility). The utility-maximized computing resource allocation problem is proved to be a COP once task offloading and assignment are determined and it is solved by the Lagrangian method and Karush-Kuhn-Tucker (KKT) conditions.
3) We develop a binary particle swarm optimization (BPSO)-based task offloading heuristic to find the best offloading decision that maximizes the utility for the system while meeting the reliability requirement with our proposed task assignment and resource allocation algorithms. Although BPSO is commonly used in the literature, we customize it for voiding falling into local optima and increasing convergence speed and integrate it with our proposed theories and advanced algorithms.
4) We perform extensive simulations and testbed experiments to validate the efficacy of our scheme in enhancing QoE and reliability. Results show that our scheme is superior to many benchmark approaches.

Note that we propose three theorems in the paper. These theorems not only provide clear guidance for our algorithm design but also introduce new perspectives and methods for task assignment and resource allocation in MU-MEC systems, which have important theoretical and practical values.

The rest of this paper is structured as follows. Section II examines closely related work and Section III elaborates system models. Section IV presents problem formulation and provides a methodology overview. Sections V-A-VII detail the proposed algorithms. Section VIII verifies the methodology. Finally, Section IX concludes the paper and discusses future work.

II. Related Works

With the emergence of computation-intensive and energy-consuming applications in IoT, offloading and scheduling tasks of MD users on ESs of an MEC system becomes an effective way to raise the QoE of users by shortening execution delay and reducing energy consumption. Task offloading and scheduling strategies for QoE optimization have been extensively studied in recent years. Targeting at independent tasks, Kuang et al. [16] designed a computation offloading and data transmission power allocation approach to mitigate latency as well as energy consumption of task execution and transmission. Wang et al. [17] developed a latency-aware task offloading algorithm that utilizes reinforcement learning to adapt fast to varying MEC system environments and adopts directed acyclic graphs to model mobile applications with data dependency. The above-mentioned two approaches are both designed for single-user MEC (SU-MEC) systems. Compared to SU-MEC systems, the task offloading and scheduling issues in MU-MEC systems are more challenging because of the intense competition among multiple MD users for limited resources. Mei et al. [24] designed a task offloading and resource allocation scheme based on iterative policy optimization strategies to minimize the energy consumption of MDs in MU-MEC systems. Chang et al. [18] presented a stochastic radio resource allocation and computation offloading method to decrease the time and energy costs of MDs in MU-MEC systems. Gupta et al. [19] solved the task offloading and resource allocation problem for maximizing an MU-MEC network's lifetime while Tran et al. [13] addressed a similar problem for optimizing an MU-MEC system's offloading performance. Chu et al. [25] addressed the allocation problem of bandwidth and computing resources in a user-game manner for MU-MEC systems to enhance their resource utilization. Guo et al. [20] proposed a Lyapunuov-guided task offloading and scheduling algorithm for multiple MDs powered by renewable energy in MU-MEC systems. To be consistent with the ever-increasing user demand, this paper is interested in dealing with the task offloading and scheduling issues in MU-MEC systems.

Although the aforementioned methods are useful to promote QoE performance, none of them takes into account reliability that is crucial to an MEC system. There are a few papers focusing on reliability optimization problems in MEC systems. Dong et al. [21] designed a reliable task offloading and assignment algorithm for MU-MEC systems. They improved the reliability

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

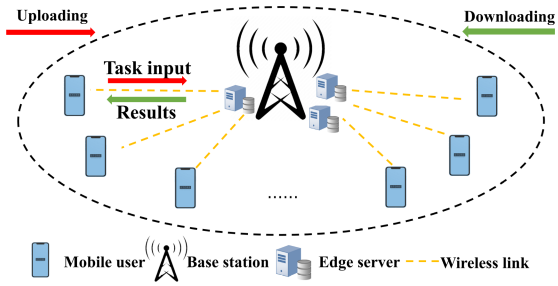


Fig. 1. Illustration of an MU-MEC system.

of tasks executed on ESs by deploying shadow servers for task re-execution in case ESs fail. Further, the authors [22] proposed a new lazy shadowing heuristic to make the best trade-off between shadow server size and task execution reliability. Liang et al. [23] studied and solved the problem of improving service reliability of an MEC network by using redundant virtualized network function instances on ESs as backups. However, all these methods ignore the reliability of data transmission used for task offloading. In addition, they would inevitably introduce extra time and energy overheads when optimizing reliability due to the use of redundant hardware and software resources.

Unlike the above-mentioned research efforts focusing on either QoE optimization or reliability augmentation, this paper deals with the QoE and reliability issues in MU-MEC systems simultaneously. More precisely, we propose a new task offloading and scheduling scheme to maximize the QoE of MU-MEC systems with a reliability constraint. In our scheme, the QoE of MD users is optimized from the aspects of latency and energy. Task execution reliability and data transmission reliability are both considered to evaluate system reliability. Through exploiting the heterogeneity of tasks and ESs, our scheme can enhance system reliability without inducing extra time and energy overheads. This work is a significantly extended version of an earlier conference paper [1]. Compared to the conference version, this paper proves the reliability-optimality of the proposed task-to-ES assignment method and the convexity of the studied computing resource allocation problem, proposes a BPSO-based task offloading method that finds the best offloading strategy for maximizing system utility with fast convergence speed, and carries out a set of testbed experiments and more simulation experiments for evaluation.

III. System Model

A. MEC System Model

The MU-MEC system consists of a BS, multiple ESs and plenty of resource-limited MDs, as illustrated in Fig. 1. The ESs could be either physical servers or virtual machines with moderate computation capability. Typically, they are deployed along with the BS for providing computation offloading services to resource-limited MDs. The BS can help resource-limited MDs complete computation-intensive tasks via offloading these tasks to ESs through the wireless channel. Let $S = \{S_1, S_2, \ldots, S_M\}$ and $D = \{D_1, D_2, \ldots, D_N\}$ represent the sets of M ESs and N MDs in the MU-MEC system. We equally divide time into discrete slots, each with a fixed length. The time slot is short such that we can safely assume each MD has one computation task as in [13], [26]. In addition, we focus on delay-sensitive tasks, which must be completed within a single time slot to meet service requirements. Thus, we can independently determine the offloading and scheduling of tasks in different time slots. For arbitrary time slot, let $\Gamma = \{\tau_1, \tau_2, \ldots, \tau_N\}$ represent the set of tasks submitted by N MD users, which can be derived at the beginning of each time slot using a workload prediction method. The characteristics of the task of device D_i $(1 \le i \le N)$ are modeled as a triple $\tau_i : \{d_i, c_i, \nu_i\}$ where d_i denotes the amount of data transferred for offloading task τ_i and c_i denotes the number of computation cycles of task τ_i to be completed. ν_i is the vulnerability factor of task τ_i, representing the probability of a transient fault occurring at the hardware level ultimately leading to a program failure [27].

Every task can be executed either on its MD or offloaded to an ES. Suppose $X = \{x_i | 1 \le i \le N\}$ denotes a task offloading strategy where $x_i = 1$ if task τ_i is offloaded to an ES and $x_i = 0$ otherwise. Given a task offloading strategy X, we can divide task set Γ into a set Γ_{loc} of tasks for local computing and a set Γ_{off} of tasks for offloading computing. In the offloading, the MU-MEC system allows multiple MDs to offload their tasks to the same ES concurrently through sharing the ES's computing resources. When an MD's task is offloaded to an ES, the ES would allocate a portion of its computing resources to complete the task and send the result back to the device. Following [13], we use the computation capacity F_j (in cycles/s) of ES S_j $(1 \le j \le M)$ to quantify the server's computing resources. Then the computing resource allocation strategy can be represented as $F = \{f_{ij} \le F_j | 1 \le i \le N, 1 \le j \le M\}$ where f_{ij} is the computing resources allocated by ES S_j to task τ_i.

B. Communication Model

Although task offloading extends the computing capability of MDs, it incurs extra communication delay: the time of transferring the input tasks (in the form of data) to MEC servers and the time of transferring the output results to MDs. Similarly, as in [13], [28], [29], [30], the time of transferring output result is ignored in the communication delay since the size of output results is significantly smaller than that of input data and the download speed is significantly higher than the upload speed. Without loss of generality, we adopt the OFDMA technology [31] in the uplink to support multiple access to ESs. To guarantee the orthogonality of upload transmissions among the MDs who use offloading services, the BS bandwidth (represented by B) is divided into equal sub-bands and every MD is allocated one. Following the Shannon-Hartley theorem [32], the data upload rate r_i from device D_i to the BS is

$$r_i = \frac{B}{n} \log_2 \left(1 + \frac{p_i g_i}{\omega + \sum_{k=1, k \ne i}^{n} p_k g_k} \right), \qquad (1)$$

where B/n is the bandwidth allocated to each of n $(n \le N)$ MDs adopting offloading services, p_i is the transmission power of device D_i, g_i is the channel gain between device D_i and the BS, and ω is the background noise. p_i and g_i are fixed once the uplink between D_i and base station is built.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

C. Computation Model

Tasks can be either executed on MDs locally or offloaded to ESs and then executed on ESs remotely. When an MD offloads its task to an ES, the MD consumes less energy for task execution at the cost of additional time and energy for data transmission.

1) Local Computing: The latency and energy consumption of executing task τ_i locally on the MD are calculated as

$$t_i^{\text{loc}} = \frac{c_i}{f_i^{\text{loc}}}, \tag{2}$$

$$E_i^{\text{loc}} = \kappa \left(f_i^{\text{loc}}\right)^2 c_i, \tag{3}$$

where f_i^{loc} is the computation capability of device D_i and κ is the processor-dependent parameter [32].

2) Offloading Computing: For task remote execution, the task should be offloaded to an ES first and then executed by the ES. In this case, the latency t_i^{off} of task τ_i is the sum of transmission delay t_i^{trans} and execution delay t_i^{exe}, i.e.,

$$t_i^{\text{off}} = t_i^{\text{trans}} + t_i^{\text{exe}} = \frac{d_i}{r_i} + \frac{c_i}{f_{ij}}. \tag{4}$$

Since task τ_i is executed on the ES, device D_i only has the energy overhead of data transmission, i.e.,

$$E_i^{\text{trans}} = p_i \times t_i^{\text{trans}}, \tag{5}$$

where p_i is the device's transmission power.

D. Reliability Model

Tasks in the system may suffer faults during transmission and execution. Thus, we model the reliability for the both.

1) Transmission Reliability: The offloaded tasks probably suffer noise, interference bit errors and bit synchronization errors over transmission links [33]. The transmission reliability of task τ_i, $\mathcal{R}_i^{\text{trans}}$, is defined as the probability that task τ_i is successfully transmitted to its offloaded ES without suffering bit errors. Following the bit error model [33], the transmission reliability of task τ_i under the bit error rate γ is derived as

$$\mathcal{R}_i^{\text{trans}} = e^{-\gamma \times t_i^{\text{trans}}}. \tag{6}$$

2) Execution Reliability: Tasks probably suffer soft errors during their executions, which are a type of failures induced by transient faults that appear temporarily and have no damage to hardware [34]. The execution reliability of task τ_i, $\mathcal{R}_i^{\text{exe}}$, is defined as the probability that task τ_i is successfully executed in the face of soft errors [35]. Since the MD and ES both use CMOS processors, the execution reliability model for CMOS processors can be applied to both. The exponential distribution is frequently used to model the execution reliability of CMOS processors. Following the exponential model widely used in the literature (e.g., [27], [35], [36], [37]), the reliability of executing task τ_i locally on the MD or ES is calculated as

$$\mathcal{R}_i^{\text{exe}} = \begin{cases} \mathcal{R}_{i,\text{loc}}^{\text{exe}} = e^{-\lambda\left(f_i^{\text{loc}}\right) \times \nu_i \times \frac{c_i}{f_i^{\text{loc}}}}, & \text{if} \quad x_i = 0 \\ \mathcal{R}_{i,\text{off}}^{\text{exe}} = e^{-\lambda(f_{ij}) \times \nu_i \times \frac{c_i}{f_{ij}}}, & \text{if} \quad x_i = 1 \end{cases}, \tag{7}$$

where f_i^{loc} is the computation capability of device D_i and f_{ij} is the computing resources allocated by ES S_j to task τ_i. $\lambda(f_i^{\text{loc}})$ and $\lambda(f_{ij})$ are the raw fault rates of D_i and S_j, respectively. Clearly, $\lambda(f_i^{\text{loc}}) \times \nu_i$ and $\lambda(f_{ij}) \times \nu_i$ are the fault rate of task τ_i executed on D_i and S_j, respectively. $\frac{c_i}{f_i^{\text{loc}}}$ and $\frac{c_i}{f_{ij}}$ are the execution time of task τ_i on D_i and S_j, respectively.

3) System Reliability: On the top of the two reliability models, the reliability of task τ_i is expressed as

$$\mathcal{R}_i = \begin{cases} \mathcal{R}_{i,\text{loc}}^{\text{exe}}, & \text{if} \quad x_i = 0 \\ \mathcal{R}_i^{\text{trans}} \times \mathcal{R}_{i,\text{off}}^{\text{exe}}, & \text{if} \quad x_i = 1 \end{cases}, \tag{8}$$

where x_i implies the offloading decision on task τ_i. The reliability of an MU-MEC system depends on the successful transmission and execution of all tasks. Thus, it is computed as the product of all independent tasks' reliability, i.e.,

$$\mathcal{R}_{\text{sys}} = \prod_{\tau_i \in \Gamma} \mathcal{R}_i. \tag{9}$$

IV. Problem Definition and Methodology Overview

A. Problem Definition

The main concern of MEC users is their QoE. According to [1], [13], [14], we know that the two most common factors impacting the users' QoE are task execution latency and device battery energy. To quantify the users' QoE of offloading and scheduling, a metric *utility* reflecting the optimization on latency and energy efficiency realized by task offloading and scheduling is introduced. Specifically, the offloading and scheduling utility of device D_i is defined as

$$\mho_i = x_i \left(\alpha_i \left(\frac{t_i^{\text{loc}} - t_i^{\text{off}}}{t_i^{\text{loc}}} \right) + \beta_i \left(\frac{E_i^{\text{loc}} - E_i^{\text{trans}}}{E_i^{\text{loc}}} \right) \right). \tag{10}$$

α_i and β_i are weighting factors implying the users' preferences on latency and energy, respectively. They are both in the range of [0,1] and hold for $\alpha_i + \beta_i = 1$. Their values can be flexibly adjusted to meet the requirements of MD users for execution latency or energy efficiency.

Considering that i) not all tasks can be offloaded to ESs because of the limited communication and computation resources and ii) tasks may suffer faults during the transmission and execution, we aim at maximizing the offloading and scheduling utility of MU-MEC systems with a given reliability constraint. We formulate the optimization problem as

$$\text{maximize:} \quad \sum_{i=1}^{N} \mho_i \tag{11}$$

$$\text{subject to:} \quad x_i = \{0, 1\}, \ \forall i = 1, 2, \ldots, N \tag{12}$$

$$f_{ij} > 0, \ \forall \tau_i \in \Gamma_{\text{off}}, \ \forall j = 1, 2, \ldots, M \tag{13}$$

$$\sum_{\tau_i \in \Gamma_j} f_{ij} \leq F_j, \ \forall j = 1, 2, \ldots, M \tag{14}$$

$$\mathcal{R}_{\text{sys}} \geq \mathcal{R}_{\text{th}}. \tag{15}$$

Eq. (12) indicates that every user could only execute its task either locally on the MD or remotely on the ES. Eqs. (13)–(14) define the lower and upper bounds on the amount of computing resources allocated to every offloaded task, where Γ_j is a subset of Γ_{off} and it contains all the tasks offloaded to ES S_j. Eq. (15)

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

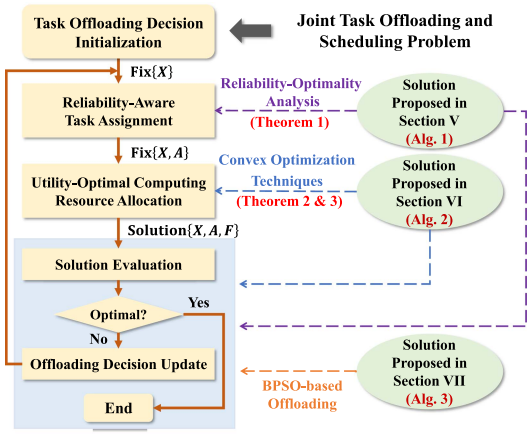


Fig. 2. Overview of our proposed scheme.

requires that the system reliability $\mathcal{R}_{\text{sys}}$ cannot be lower than a threshold $\mathcal{R}_{\text{th}}$.

B. Methodology Overview

To solve the task offloading and scheduling problem, we need to determine i) the tasks to be offloaded, ii) the task-to-ES assignment, and iii) the allocation of computing resources to tasks offloaded on ESs. Clearly, this problem is a COP involving three decision variables, which is difficult to solve directly. We solve the problem as follows. First, we initialize the task offloading policy X at random. Given the offloading policy X which determines the task set Γ_{off} to be offloaded, we propose a reliability-aware task assignment algorithm A to judiciously assign the tasks of set Γ_{off} to ESs for enhancing system reliability. After deriving the task offloading and assignment strategies $\{X, A\}$, we formulate the utility maximization problem as a COP and address the COP to obtain the computing resource allocation solution F using the Lagrangian method and KKT conditions. Last, we evaluate the quality of solution $\{X, A, F\}$ and design a BPSO-based algorithm to iteratively find the best task offloading decision that can achieve the global optimum system QoE under the reliability constraint using our task assignment and resource allocation methods. To ease the understanding of the proposed scheme, we show its high-level overview in Fig. 2.

V. Reliability-Aware Task Assignment

Although hardware and software redundancy based fault tolerance techniques are very useful to enhance reliability, they are in general time/energy-consuming and thus may be not suitable for time/energy-critical systems. In this paper, for a given set of tasks to be offloaded, we propose a task-to-ES assignment scheme which would not have extra time/energy overhead in improving system reliability by fully exploiting the heterogeneity of tasks and ESs.

A. Reliability-Optimality Analysis of Task Assignment

Our task-to-ES assignment scheme is motivated by [35]. However, there are many differences (e.g., the formulation of system reliability as well as the design and implementation of heuristic algorithm) between our scheme and the method in [35]. Given an offloading policy X that divides the task set Γ into the local task set Γ_{loc} and the offloaded task set Γ_{off}, we can re-write the system reliability in (9) as

$$\begin{aligned}\mathcal{R}_{\text{sys}} &= \prod_{\tau_i\in\Gamma}\mathcal{R}_i = \prod_{\tau_i\in\Gamma_{\text{loc}}}\mathcal{R}^{\text{exe}}_{i,\text{loc}} \times \prod_{\tau_i\in\Gamma_{\text{off}}}(\mathcal{R}^{\text{trans}}_i \times \mathcal{R}^{\text{exe}}_{i,\text{off}}) \\ &= \prod_{\tau_i\in\Gamma_{\text{loc}}}\mathcal{R}^{\text{exe}}_{i,\text{loc}} \times \prod_{\tau_i\in\Gamma_{\text{off}}}\mathcal{R}^{\text{trans}}_i \times \prod_{\tau_i\in\Gamma_{\text{off}}}\mathcal{R}^{\text{exe}}_{i,\text{off}} \\ &= \mathcal{R}^{\text{exe}}_{\text{loc}} \times \mathcal{R}^{\text{trans}}_{\text{off}} \times \mathcal{R}^{\text{exe}}_{\text{off}} = e^{\left(-\sum_{\tau_i\in\Gamma_{\text{loc}}}\lambda\left(f_i^{\text{loc}}\right)\times\nu_i\times\frac{c_i}{f_i^{\text{loc}}}\right)} \\ &\quad \times e^{\left(-\sum_{\tau_i\in\Gamma_{\text{off}}}\gamma\times\frac{d_i}{r_i}\right)} \times e^{\left(-\sum_{\tau_i\in\Gamma_{\text{off}}}\lambda(f_{ij})\times\nu_i\times\frac{c_i}{f_{ij}}\right)}.\end{aligned} \tag{16}$$

Clearly, $\mathcal{R}_{\text{sys}}$ is the product of $\mathcal{R}^{\text{exe}}_{\text{loc}}$, $\mathcal{R}^{\text{trans}}_{\text{off}}$, and $\mathcal{R}^{\text{exe}}_{\text{off}}$. The 1st item $\mathcal{R}^{\text{exe}}_{\text{loc}} = \prod_{\tau_i\in\Gamma_{\text{loc}}}\mathcal{R}_i$ is the execution reliability of local tasks in Γ_{loc}. It is constant once Γ_{loc} is derived since all parameters are related to either local tasks or the given MDs. The 2nd item $\mathcal{R}^{\text{trans}}_{\text{off}} = \prod_{\tau_i\in\Gamma_{\text{off}}}\mathcal{R}^{\text{trans}}_i$ is the transmission reliability of offloaded tasks in Γ_{off}. In this item, γ is a constant and d_i is a task-related parameter. r_i is mainly determined by the number n of MDs adopting task offloading, the transmission power p_i and the channel gain g_i. The parameters n, p_i, g_i are all fixed once Γ_{off} is decided. Thus we can conclude that given an offloading policy X, $\mathcal{R}_{\text{sys}}$ is decided by the 3rd item $\mathcal{R}^{\text{exe}}_{\text{off}}$ and it is maximal when $\mathcal{R}^{\text{exe}}_{\text{off}}$ is maximal. That's to say, maximizing $\mathcal{R}_{\text{sys}}$ is equivalent to maximizing $\mathcal{R}^{\text{exe}}_{\text{off}}$.

Consider an ideal case that every task τ_i of set Γ_{off} executes at its offloaded ES's maximal capacity F_j. The reliability $\mathcal{R}^{\text{exe}}_{\text{off}}$ in such case is derived as

$$\begin{aligned}\mathcal{R}^{\text{exe}}_{\text{off}} &= \prod_{\tau_i\in\Gamma_{\text{off}}}\mathcal{R}^{\text{exe}}_{i,\text{off}}(F_j) = e^{\left(-\sum_{\tau_i\in\Gamma_{\text{off}}}\lambda(F_j)\times\nu_i\times\frac{c_i}{F_j}\right)} \\ &= e^{\left(-\sum_{j=1}^{M}\frac{\lambda(F_j)}{F_j}\times(\sum_{\tau_i\in\Gamma_j}\nu_i\times c_i)\right)} \\ &= e^{(-(y_1\times z_1+y_2\times z_2+\cdots+y_M\times z_M))} = e^{(-\mathbf{y}\cdot\mathbf{z})}.\end{aligned} \tag{17}$$

Through the above algebra transformation, $\mathcal{R}^{\text{exe}}_{\text{off}}$ is decided by the product of vectors $\mathbf{y}$ and $\mathbf{z}$ where $y_j = \frac{\lambda(F_j)}{F_j} \in \mathbf{y}$ and $z_j = \sum_{\tau_i\in\Gamma_j}\nu_i \times c_i \in \mathbf{z}$ denote the vulnerability index of ES S_j and task set Γ_j, respectively.[1] We have the following observations on vectors $\mathbf{y}$ and $\mathbf{z}$ with respect to reliability.

1) The sum of $\nu_i \times c_i$ of all tasks in the given offloaded task set Γ_{off} is a constant, i.e., $z_1 + z_2 + \cdots + z_M$ is a constant.
2) A task/ES having a relatively smaller vulnerability index means that it is relatively more reliable.
3) Given an MU-MEC system, $\mathbf{y}$ is fixed while $\mathbf{z}$ is not and is decided by task-to-ES assignment.

[1]The server's vulnerability index $y_j = \frac{\lambda(F_j)}{F_j}$ reflects the server's failure rate. A higher frequency results in a lower fault rate. Therefore, both a higher frequency F_j and a lower fault rate $\lambda(F_j)$ lead to smaller y_j, indicating that the server is less vulnerable to soft errors. On the other hand, the task's vulnerability index $\nu_i \times c_i$ is the product of ν_i and c_i. Clearly, a higher vulnerability factor ν_i and an increased execution cycles c_i lead to a larger $\nu_i \times c_i$, indicating that the task is more vulnerable to soft errors.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

On the basis of these observations, we propose a theorem (i.e., Theorem 1) on task-to-ES assignment which obtains the minimized $\mathbf{y}\cdot\mathbf{z}$ for maximizing $\mathcal{R}_{\text{off}}^{\text{exe}}$ and thus maximizing $\mathcal{R}_{\text{sys}}$. Note that despite the assumption $f_{ij}=F_j$ used in the above analysis, we can still have these observations and then deduce Theorem 1 for any other cases of f_{ij} once f_{ij} is determined.

Theorem 1: Given a task offloading policy X that decides the tasks to be assigned, if the ES vulnerability index y_i in $\mathbf{y}$ meet $y_1\le y_2\le\cdots\le y_M$ and the sum of task set vulnerability index z_j in $\mathbf{z}$ is constant, then $\mathbf{y}\cdot\mathbf{z}$ is minimal if $z_1\ge z_2\ge\cdots\ge z_M$ holds. At this time, the system reliability $\mathcal{R}_{sys}$ is maximized.

Proof: Given the offloaded tasks to be assigned to M ESs, let $\mathbf{z}^*=[z_1^*,z_2^*,\ldots,z_j^*,\ldots,z_M^*]^T$ denote the reliability-optimum task assignment method where z_j^* is the vulnerability index of task set Γ_j assigned to ES S_j, and $\mathcal{R}_{\text{sys}}^*$ denote the optimum system reliability achieved by $\mathbf{z}^*$. We can derive another task assignment method $\mathbf{z}'$ by randomly swapping two elements in $\mathbf{z}^*$. Assuming z_a^* and z_b^* $(1\le a<b\le M)$ in $\mathbf{z}^*$ are swapped, $\mathbf{z}^*$ then becomes $\mathbf{z}'=[z_1^*,z_2^*,\ldots,z_{a-1}^*,z_b^*,z_{a+1}^*,\ldots,z_{b-1}^*,z_a^*,z_{b+1}^*,\ldots,z_M^*]^T$. Correspondingly, let $\mathcal{R}'_{\text{sys}}$ denote the system reliability achieved by $\mathbf{z}'$. According to the reliability model defined in (17), we have

$$\frac{\mathcal{R}_{\text{sys}}^*}{\mathcal{R}'_{\text{sys}}}=\frac{\exp\{-\mathbf{y}\cdot\mathbf{z}^*\}}{\exp\{-\mathbf{y}\cdot\mathbf{z}'\}}=\exp\{-y_a\cdot z_a^*-y_b\cdot z_b^*+y_a\cdot z_b^*+y_b\cdot z_a^*\}=\exp\{(y_a-y_b)(z_b^*-z_a^*)\}. \tag{18}$$

As $\mathcal{R}_{\text{sys}}^*$ is the optimum system reliability, we can deduce that $\frac{\mathcal{R}_{\text{sys}}^*}{\mathcal{R}'_{\text{sys}}}\ge 1$. In other words, we have $(y_a-y_b)(z_b^*-z_a^*)\ge 0$. The ES vulnerability index in $\mathbf{y}$ satisfy $y_1\le y_2\le\cdots\le y_M$, thus $y_a\le y_b$ holds for $a<b$. In the end, $z_a^*\ge z_b^*$ is derived. From the above analysis, we find that for any two ESs of the optimum task assignment, the ES having smaller ES vulnerability index is assigned a task set having larger task set vulnerability index.

Furthermore, any feasible task assignment method can be obtained through multiple exchanges of elements in $\mathbf{z}^*$. We can prove that $z_a^*\ge z_b^*$ where $a<b$ is true in each exchange as analyzed above. In conclusion, when the ES with smaller vulnerability index accommodates the set of its assigned tasks with larger vulnerability index, i.e., $y_1\le y_2\le\cdots\le y_M$ and $z_1\ge z_2\ge\cdots\ge z_M$, the system reliability is maximized. In other words, given the tasks to be offloaded, the MEC system's reliability is maximal when unreliable tasks are executed on reliable ESs. ■

B. Reliability-Aware Task Assignment Algorithm

Following Theorem 1, we develop a heuristic to assign unreliable tasks having relatively larger vulnerability to reliable ESs having relatively smaller vulnerability index. Algorithm 1 shows the pseudo-code of our heuristic. Given the tasks Γ_{off} to be offloaded and the ES system S, the algorithm first initializes the task sets $\{\Gamma_1,\ldots,\Gamma_M\}$ assigned to M ESs as empty (lines 1-2) and sorts the tasks in Γ_{off} in decreasing order according to vulnerability index $\nu_i\times c_i$ (lines 3-4). It then constructs the M task sets by iteratively assigning tasks with relatively larger vulnerability indices to the ESs having relatively smaller vulnerability indices (lines 5-9). In this way, the task set with the largest vulnerability is assigned to the ES having the smallest vulnerability, and the task set with the next largest vulnerability is assigned to the ES having the next smallest vulnerability. The algorithm stops the iterations until all tasks in Γ_{off} are assigned and outputs the assignment strategy A.

Algorithm 1: Reliability-Aware Task Assignment.

Input: Set of tasks to be offloaded, represented by Γ_{off} and M ESs $S=\{S_1,S_2,\cdots,S_M\}$ meeting $y_1\le y_2\le\cdots\le y_M$
Output: Task-to-ES assignment strategy $A=\{\Gamma_1,\Gamma_2,\cdots,\Gamma_M\}$

1 **for** $j=1$ to M **do**
2 initialize the set Γ_j of tasks assigned to ES S_j as $\emptyset$;
3 **for** $i=1$ to $|\Gamma_{\text{off}}|$ **do**
4 sort tasks in Γ_{off} in decreasing order according to $\nu_i\times c_i$;
5 $j=1, i=1$;
6 **while** $i\le|\Gamma_{\text{off}}|$ **do**
7 $\Gamma_j=\Gamma_j+\tau_i, i++, j++$;
8 **if** $j>M$ **then**
9 $j=1$;

VI. Utility-Optimum Resource Allocation

This section formulates and solves the computing resource allocation problem for all tasks offloaded on ESs with the goal of maximizing system utility.

A. Convex Optimization Problem

Substituting (10) into (11), we have

$$\begin{aligned}\sum_{\tau_i\in\Gamma_{\text{off}}}\mho_i&=\sum_{j=1}^{M}\sum_{\tau_i\in\Gamma_j}\mho_i\\&=\sum_{j=1}^{M}\sum_{\tau_i\in\Gamma_j}\left(1-\frac{\alpha_i d_i f_i^{\text{loc}}}{r_i c_i}-\frac{\beta_i p_i d_i}{r_i\kappa\left(f_i^{\text{loc}}\right)^2 c_i}\right)\\&\quad-\sum_{j=1}^{M}\sum\nolimits_{\tau_i\in\Gamma_j}\frac{\alpha_i f_i^{\text{loc}}}{f_{ij}},\end{aligned} \tag{19}$$

where p_i and r_i are the transmit power and transmission rate of device D_i. The both are known after task-to-ES assignment. Therefore, the 1st item of (19) is constant and then the original utility maximization problem defined in (11)–(15) could be reduced to

$$\begin{aligned}\text{minimize:}\quad&\varphi(f_{ij})=\sum_{j=1}^{M}\sum\nolimits_{\tau_i\in\Gamma_j}\frac{\alpha_i f_i^{\text{loc}}}{f_{ij}}\\\text{subject to:}\quad&\sum_{\tau_i\in\Gamma_j}f_{ij}\le F_j,\ \forall j=1,2,\ldots,M,\\&f_{ij}>0,\ \forall\tau_i\in\Gamma_j,\ \forall j=1,2,\ldots,M.\end{aligned}$$

Here the reliability constraint is neglected because it is very likely to be met by using the proposed task-to-ES assignment method and the constraint will be examined after the computing resource allocation. It is worth noting that the process of allocating computing resources on different servers is independent

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

of each other. Thus, we can focus on optimizing the allocation of computing resources on individual servers, which can be represented as

$$\text{minimize:} \quad \varphi_j\left(f_{ij}\right) = \sum_{\tau_i \in \Gamma_j} \frac{\alpha_i f_i^{\text{loc}}}{f_{ij}} \tag{20}$$

$$\text{subject to:} \quad \sum_{\tau_i \in \Gamma_j} f_{ij} \le F_j, \ \forall j = 1, 2, \ldots, M, \tag{21}$$

$$f_{ij} > 0, \ \forall \tau_i \in \Gamma_j, \ \forall j = 1, 2, \ldots, M. \tag{22}$$

where $\varphi(f_{ij}) = \sum_{j=1}^{M} \varphi_j(f_{ij})$ and $\varphi(f_{ij})$ is minimized when the each $\varphi_j(f_{ij})$ of ES S_j is minimized. Obviously, the reduced problem is a function of f_{ij} that represents the computing resource allocated by ES S_j to task τ_i. Our objective is to find an optimum resource allocation strategy for minimizing $\varphi_j(f_{ij})$ and hence maximizing the utility $\sum_{i=1}^{N} \mho_i$. To this end, Theorem 2 is proposed to show that the reduced problem is a COP.

Theorem 2: The computing resource allocation problem formulated in (20)–(22) is a COP.

Proof: The Hessian matrix of objective function $\varphi_j(f_{ij})$ of the resource allocation problem in (20)–(22) is derived by computing the 2nd-order derivative of $\varphi_j(f_{ij})$ as

$$\frac{\partial^2 \varphi_j\left(f_{ij}\right)}{\partial f_{ij}^2} = \frac{2\alpha_i f_i^{\text{loc}}}{f_{ij}^3} > 0, \ \forall \tau_i \in \Gamma_j, \tag{23}$$

$$\frac{\partial^2 \varphi\left(f_{ij}\right)}{\partial f_{ij} \partial f_{mn}} = 0, \ \text{if}(i,j) \ne (m,n). \tag{24}$$

Since all the diagonal elements of the Hessian matrix are greater than 0, the matrix is positive definite. Furthermore, the constraints in (21)–(22) are convex. Therefore, according to the definition of a convex problem, the formulated problem is a COP. ■

B. Optimum Resource Allocation Algorithm

To address the COP, we first use the Lagrangian multiplier method to construct the Lagrangian function of (20)–(22) and then utilize the KKT conditions to solve the function. Through this, we have the following theorem on the optimum computing resource allocation strategy F^*.

Theorem 3: Using the Lagrangian multiplier method and KKT conditions, the optimum computing resource allocation solution f_{ij}^* to the convex problem given in (20)–(22) and the minimum value of $\varphi_j(f_{ij})$ can be derived as

$$f_{ij}^* = \frac{F_j \sqrt{\alpha_i f_i^{\text{loc}}}}{\sum_{\tau_i \in \Gamma_j} \sqrt{\alpha_i f_i^{\text{loc}}}}, \forall \tau_i \in \Gamma_j, 1 \le j \le M, \tag{25}$$

$$\varphi_j\left(f_{ij}^*\right) = \frac{1}{F_j} \left(\sum\nolimits_{\tau_i \in \Gamma_j} \sqrt{\alpha_i f_i^{\text{loc}}} \right)^2. \tag{26}$$

Proof: Let $L(\varphi_j(f_{ij}), \upsilon_j)$ be the Lagrangian function of the problem formulated in (20)–(22). It is expressed as

$$L\left(\varphi_j\left(f_{ij}\right), \upsilon_j\right) = \sum_{\tau_i \in \Gamma_j} \frac{\alpha_i f_i^{\text{loc}}}{f_{ij}} + \upsilon_j \left(\sum_{\tau_i \in \Gamma_j} f_{ij} - F_j \right), \tag{27}$$

Algorithm 2: Utility-Optimum Resource Allocation.

Input: Task-to-ES assignment strategy generated by **Algorithm 1**, $A = \{\Gamma_1, \Gamma_2, \cdots, \Gamma_M\}$
Output: Utility-optimum resource allocation strategy $F^* = \{f_{ij}^* | \ \forall \tau_i \in \Gamma_j, \ \forall j = 1, 2, \cdots, M\}$

```
for j = 1 to M do
    for i = 1 to |Γ_j| do
        compute the optimum computing resource f*_ij
          allocated to task τ_i on ES S_j by Eq. (25);
        f_ij = f*_ij;
```

where υ_j is the Lagrange multiplier vector. First, taking the derivative of $L(\varphi_j(f_{ij}), \upsilon_j)$ and setting its derivative to 0, we have

$$\frac{\partial L\left(\varphi_j\left(f_{ij}, \upsilon_j\right)\right)}{\partial f_{ij}} = -\frac{\alpha_i f_{ij}}{f_{ij}^2} + \upsilon_j = 0. \tag{28}$$

Solving (28), the optimum computing resource allocation strategy f_{ij}^* is deduced as

$$f_{ij}^* = \sqrt{\frac{\alpha_i f_i^{\text{loc}}}{\upsilon_j}}, \tau_i \in \Gamma_j, \tag{29}$$

where υ_j (> 0) meets the complementary relaxation condition

$$\sum_{\tau_i \in \Gamma_j} f_{ij}^* - F_j = 0, \tag{30}$$

According to (29) and (30), we get

$$\upsilon_j = \left(\frac{\sum_{\tau_i \in \Gamma_j} \sqrt{\alpha_i f_i^{\text{loc}}}}{F_j} \right)^2. \tag{31}$$

Then, the optimum strategy f_{ij}^* can be obtained by substituting (31) into (29). It is expressed as

$$f_{ij}^* = \frac{F_j \sqrt{\alpha_i f_i^{\text{loc}}}}{\sum_{\tau_i \in \Gamma_j} \sqrt{\alpha_i f_i^{\text{loc}}}}, \forall \tau_i \in \Gamma_j. \tag{32}$$

Finally, substituting (32) into (20), we derive the optimum objective function as

$$\varphi_j\left(f_{ij}^*\right) = \frac{1}{F_j} \left(\sum_{\tau_i \in \Gamma_j} \sqrt{\alpha_i f_i^{\text{loc}}} \right)^2. \tag{33}$$

■

Theorem 3 is applicable for the allocation of computing resources on each server. Therefore, we develop a convex optimization-based algorithm to derive the utility-optimal computing resource allocation solution F^*. Algorithm 2 summarizes the pseudo-code of our algorithm.

VII. BPSO-Based Task Offloading

Using our proposed task assignment and resource allocation methods, we can maximize system reliability and offloading utility for any feasible offloading solutions. This section focuses on finding out the best offloading solution that can obtain the global maximum system offloading utility under the reliability constraint, combined with our task assignment and resource allocation methods.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

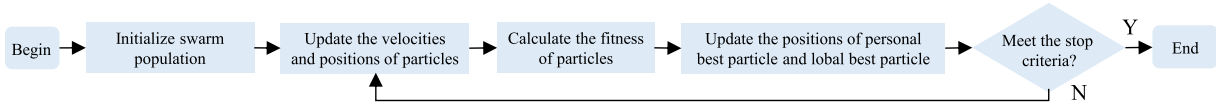


Fig. 3. Overview of BPSO.

A. Binary Particle Swarm Optimization

Clearly, the studied task offloading problem is a typical binary combinational optimization problem since each element in the offloading policy X is either 0 or 1. BPSO has been widely used in solving binary combinational optimization problems [38] since it has a larger probability of not being trapped in local optima compared to traditional heuristics and doesnot need to map continuous variables to discrete variables [39]. Thus, we apply BPSO to cope with the task offloading problem. Fig. 3 shows the flowchart of BPSO, which is the same as that of standard PSO. The main difference between standard PSO and BPSO is their functions used for updating particles' velocity and position [38], [39], [40], [41]. We introduce the velocity and position update functions used in our proposed BPSO as follows.

In our proposed BPSO-based task offloading algorithm, the swarm $\mathcal{P}$ contains K particles and the searching space is N-dimensional. Every particle is associated with a position vector $\mathbf{p}_k = (p_{k,1}, \ldots, p_{k,i}, \ldots, p_{k,N})$ and a velocity vector $\mathbf{v}_k = (v_{k,1}, \ldots, v_{k,i}, \ldots, v_{k,N})$. For the k-th $(1 \leq k \leq K)$ particle $(\mathbf{p}_k, \mathbf{v}_k)$, its position vector $\mathbf{p}_k$ represents a feasible offloading solution in which $p_{k,i} = 1$ if task τ_i is offloaded for remote execution and $p_{k,i} = 0$ otherwise. The velocity vector $\mathbf{v}_k$ is utilized to iteratively adjust the movement direction and speed of particle $(\mathbf{p}_k, \mathbf{v}_k)$ so that the particle can be updated to move towards its personal optimum position $\mathbf{p}_k^{\text{best}} = (p_{k,1}^{\text{best}}, \ldots, p_{k,i}^{\text{best}}, \ldots, p_{k,N}^{\text{best}})$. Through L round of iterations, the best position of any particles in the search space is $\mathbf{g}^{\text{best}} = (g_1^{\text{best}}, \ldots, g_i^{\text{best}}, \ldots, g_N^{\text{best}})$.

In the ℓ-th iteration, the particle's velocity $\mathbf{v}_k(\ell)$ and its new position $\mathbf{p}_k(\ell)$ will be updated using

$$\begin{aligned} v_{k,i}(\ell) = {} & \varpi \times v_{k,i}(\ell-1) \\ & + \zeta_1 \eta_1 \times \left(p_{k,i}^{\text{best}}(\ell-1) - p_{k,i}(\ell-1)\right) \\ & + \zeta_2 \eta_2 \times \left(g_i^{\text{best}}(\ell-1) - p_{k,i}(\ell-1)\right), \end{aligned} \tag{34}$$

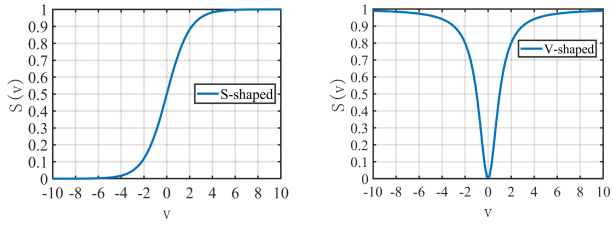
$$p_{k,i}(\ell) = p_{k,i}(\ell-1) + v_{k,i}(\ell), \tag{35}$$

where ϖ is the inertia factor, ζ_1 and ζ_2 are learning factors, η_1 and η_2 are variables taking values between $(0, 1)$ randomly. Eqs. (34)–(35) are conventional update methods widely used in PSO. However, the position of particles in BPSO cannot be updated by (35) due to the discrete binary space. To solve this problem, a transfer function is introduced in BPSO to map velocity values to probability values for updating the positions. Kennedy and Ebehart [41] designed a sigmoid transfer function which is expressed as

$$S\left(v_{k,i}(\ell)\right) = \frac{1}{1 + e^{-v_{k,i}(\ell)}}. \tag{36}$$

Then the ℓ-th particle's position is updated by

$$p_{k,i}(\ell) = \begin{cases} 1, & \text{if} \quad \text{rand}() < S\left(v_{k,i}(\ell)\right) \\ 0, & \text{if} \quad \text{rand}() \geq S\left(v_{k,i}(\ell)\right) \end{cases}, \tag{37}$$



(a) S-shaped (b) V-shaped

Fig. 4. Example of two transfer functions.

where rand() is a function generating a value between 0 and 1 at random.

As discussed in [40], the transfer functions used in BPSO are generally classified into two categories: S-shaped and V-shaped. For better understanding, Fig. 4 illustrates an example of the two functions. As can be seen, in the S-shaped transfer function based BPSO, if the velocity's value is positive, it has bigger probability of updating the next particle position to 1; and if the velocity's value is negative, it has bigger probability of updating the next particle position to 0. That's to say, different directions of velocity have impacts on the next particle's position. But in an ideal BPSO, there should be no difference between the values of velocity in positive or negative directions [38]. To overcome the drawback of S-shaped transfer function based BPSO, V-shaped transfer function based BPSO is introduced. It has been observed in [40] that the V-shaped transfer function based BPSO outperforms the S-shaped transfer function based BPSO in voiding falling into local optima and increasing convergence speed. As a consequence of the above, we use the V-shaped transfer function based BPSO to address the task offloading problem and design a new V-shaped transfer function as

$$S\left(v_{k,i}(\ell)\right) = \frac{v_{k,i}^2(\ell)}{1 + v_{k,i}^2(\ell)}. \tag{38}$$

In our proposed BPSO, the particle's velocity and position are updated by (38) and (37), respectively.

B. BPSO-Based Task Offloading Algorithm

The pseudo-code of our BPSO-based task offloading algorithm is presented in Algorithm 3. The algorithm firstly initializes all the K particles in the swarm randomly, sets every particle as its personal best particle by $\mathbf{p}_k^{\text{best}} = \mathbf{p}_k$, and calculates the fitness value (i.e., system offloading utility $\mho_k$) of each particle (lines 1-5). Afterwards, the global best position $\mathbf{g}^{\text{best}}$ is initialized as the $\mathbf{p}_k$ with the maximum $\mho_k$ (line 6). The algorithm then iteratively explores the optimum task offloading solution (lines 7-21). In each iteration, for the k-th particle, the algorithm first updates the particle's position and velocity vector using (37)–(38) (line 9). According to the newly updated position vector $\mathbf{p}_k$, we can divide the task set $\mathit{\Gamma}$ into $\mathit{\Gamma}_{\text{loc}}$ and $\mathit{\Gamma}_{\text{off}}$ (line 10).

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: BPSO-Based Task Offloading.

Input: The task set Γ of MDs
Output: The optimum offloading strategy X^*
1 **for** $k = 1$ to K **do**
2 initialize the particle $(\mathbf{p}_k, \mathbf{v}_k)$ at random;
3 set every particle as its personal best particle by $\mathbf{p}_k^{\text{best}} = \mathbf{p}_k$;
4 compute the fitness value $\mho_k$ of $\mathbf{p}_k$ by Eq. (19);
5 select $\mathbf{p}_k$ with the maximum $\mho_k$ as the global best position by $\mathbf{g}^{\text{best}} = \mathbf{p}_k$;
6 **for** $\ell = 1$ to L **do**
7 **for** $k = 1$ to K **do**
8 update the particle $(\mathbf{p}_k(\ell), \mathbf{v}_k(\ell))$ using Eqs. (37)-(38);
9 divide task set Γ into Γ_{loc} and Γ_{off} according to $\mathbf{p}_k(\ell)$;
10 assign tasks in Γ_{off} to ESs to increase system reliability $\mathcal{R}_{\text{sys}}$ using **Algorithm 1**;
11 derive the optimum resource f_{ij}^* allocated to task τ_i on ES S_j using **Algorithm 2**;
12 compute the fitness value $\mho_k(\ell)$ of $\mathbf{p}_k(\ell)$ by Eq. (19);
13 **if** $\mho_k(\ell)$ is larger than the fitness value of $\mathbf{p}_k^{\text{best}}$ **then**
14 $\mathbf{p}_k^{\text{best}} = \mathbf{p}_k(\ell)$;
15 **if** $\mho_k(\ell)$ is larger than the fitness value of $\mathbf{g}^{\text{best}}$ **then**
16 $\mathbf{g}^{\text{best}} = \mathbf{p}_k(\ell)$ and $\mho^{\text{best}} = \mho_k(\ell)$;
17 **if** $\mathbf{g}^{\text{best}}$ meets the reliability constraint in Eq. (15) **then**
18 $X^* = \mathbf{g}^{\text{best}}$;

TABLE I
MEC SYSTEM SETUP

MEC system setup	Bandwidth (MHz)	Computing capacity of 3 servers (GHz)
Low-resource	15	8, 10, 12
Medium-resource	25	14, 16, 18
High-resource	35	20, 22, 24

Then, we can calculate the fitness value $\mho_k$ of the k-th particle after determining task assignment and resource allocation using Algorithms 1 and 2 (lines 11-13). If $\mho_k$ is larger than that of the particle's previous personal optimum position, the particle's personal optimum position $\mathbf{p}_k^{\text{best}}$ is updated (lines 14-16). Meanwhile, if $\mho_k$ is also larger than that of the current global optimum position $\mathbf{g}^{\text{best}}$, the $\mathbf{g}^{\text{best}}$ and its utility $\mho^{\text{best}}$ are updated accordingly (lines 17-19). The iteration stops when a pre-defined iteration number L is reached. At last, the algorithm checks if $\mathbf{g}^{\text{best}}$ satisfies the system reliability constraint (line 22). If yes, we can derive the optimum offloading decision X^* as $\mathbf{g}^{\text{best}}$ (line 23).

VIII. EVALUATION

A. Evaluation Setups

Simulation Setup: Three simulated MU-MEC systems are used to verify the proposed scheme. Every simulated MU-MEC system is deployed with a BS and three ESs for providing offloading services to MDs. The three simulated MU-MEC systems are composed of 20 MDs, 35 MDs, and 50 MDs, respectively. Without loss of generality, we adopt three different (i.e., low, medium, and high) resource settings for every simulated MU-MEC system. As shown in Table I, the BS bandwidth B of the three resource settings are 15 MHz, 25 MHz and 35 MHz, respectively. The computation capacity F_j of three ESs in the low-resource scenario are 8 GHz, 0 GHz and 12 GHz, respectively. The computation capacity F_j of three ESs in the medium-resource scenario, are 14 GHz, 16 GHz and 18 GHz, respectively. The computation capacity F_j of three ESs in the high-resource scenario are 20 GHz, 22 GHz and 24 GHz, respectively. The settings of other key parameters are presented in Table II. We vary the MDs' computation capability f_i^{loc} in the range of [1.0, 1.2] GHz and let the MDs' transmission power p_i be distributed uniformly over [0.1, 0.5] W. The energy coefficient κ takes the value of 5×10^{-27} [13]. The parameters $\{d_i, c_i, \nu_i\}$ of every task τ_i are distributed uniformly over $[200, 1000]$ KB, $[500, 2500]$Megacycles, and $[0, 1]$, respectively. We set the background noise ω as -100 dBm. The preferences of users on latency and energy (i.e., α_i and β_i) take the value from $[0, 1]$ and hold for $\alpha_i + \beta_i = 1$. The system reliability threshold $\mathcal{R}_{\text{th}}$ is set to 0.9.

TABLE II
PARAMETER SETUPS

Notation	Parameter	Value
f_i^{loc}	Computation capability of MDs	$[1.0, 1.2]$GHz
p_i	Transmission power of MDs	$[0.1, 0.5]$W
κ	Energy coefficient	5×10^{-27}
d_i	Data size of task τ_i	$[200, 1000]$KB
c_i	Number of execution cycles of task τ_i	$[500, 2500]$ Megacycles
ν_i	Vulnerability factor of task τ_i	$[0, 1]$
ω	Background noise	-100dBm
α_i	User's preference on latency	$[0, 1]$
β_i	User's preference on energy	$[0, 1]$
$\mathcal{R}_{\text{th}}$	System reliability threshold	0.9

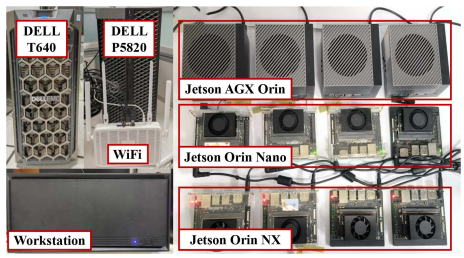


Fig. 5. A snapshot of the implemented testbed.

Testbed Experiment Setup: We also conduct numerous experiments on a hardware testbed platform to validate the performance of the proposed methodology in terms of latency and utility under different parameter settings. The platform, as shown in Fig. 5, consists of three main components: three servers as ESs, twelve Nvidia edge computing boards as MDs, and a router as the BS. Their detailed configurations are listed in Table III. The twelve boards include three device models: Jetson Orin Nano, Orin NX, and AGX Orin [42], each with a different maximum CPU frequency. For the same device model, the heterogeneity of MDs is maintained by scaling their operating frequency. All devices are connected to a router, with the MDs connected via the wireless channel and the ESs connected via the Ethernet. As shown in Table IV, we use the image detection algorithm YOLOv5 [43] to generate the instances of tasks by varying the checkpoint model and image resolution adopted by YOLOv5. The generated tasks are either locally processed on

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

TABLE III
TESTBED SETUP

Type	Device model	CPU	Type	Freq. (MHz)
End	Jetson Orin Nano	6-core ARM Cortex-A78AE	1	1510.4
			2	1267.2
			3	960.0
			4	652.8
	Jetson Orin NX	6-core ARM Cortex-A78AE	5	1984.0
			6	1728.0
			7	1420.8
			8	8832.0
	Jetson AGX Orin	12-core ARM Cortex-A78AE	9	2201.6
			10	1958.4
			11	1651.2
			12	1190.4
Edge	DELL P5820 Workstation	Intel i9-10900X	1	3500.0
		Intel i7-13700KF	2	3400.0
	DELL T640	Intel Silver 4210R	3	2000.0

[1] The twelve types of boards can be extended to more MDs in the experiment.

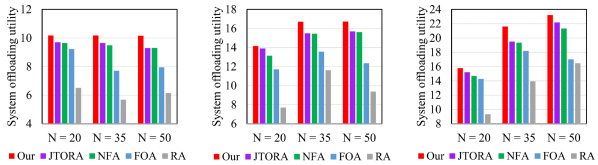
TABLE IV
BENCHMARK SETUP

Type	Checkpoint model	Type	Image resolution
1	YOLOv5n	1	640x
2	YOLOv5s	2	960x
3	YOLOv5m	3	1280x
4	YOLOv5l	4	1600x

MDs or offloaded to ESs based on the designed offloading and scheduling strategy.

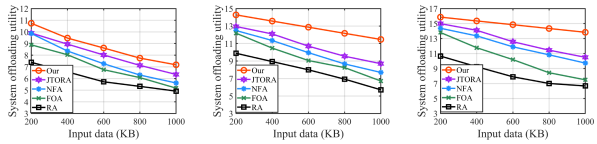
Benchmark Algorithms: We compare our scheme with four benchmark algorithms. Although the objective of these benchmark algorithms may not be the same as ours, they can also be used to solve our problem.

- *JTORA* [13] is a state-of-the-art approach used in solving the task offloading and resource allocation problem with a similar objective of optimizing latency and energy consumption. It first attempts to find the optimum solution using MINLP, and then uses quasi-convex, convex optimization, and binary search techniques to quickly derive a sub-optimum solution.
- *NFA* [44] is a new firefly algorithm. The traditional firefly algorithm (FA) is a heuristic optimization algorithm that searches for the optimal solution by simulating the brightness and movement behavior of fireflies. Each firefly's location represents a potential solution, with brightness evaluating the quality of the solution. Fireflies with relatively lower brightness would be attracted by relatively brighter fireflies and move toward them. The movement of fireflies represents the evolution of the solutions. Compared to traditional FA, NFA finds the optimum solution more efficiently by generating the best initial solution and accelerating the firefly (i.e., solution) evolution. It also can be applied to any combinational optimization problem.
- *FOA* is a widely used baseline algorithm in MEC. It offloads all tasks of MDs to ESs for remote execution.
- *RA* is also a commonly accepted baseline algorithm in MEC. In this algorithm, all the decisions on task offloading and scheduling are made randomly.



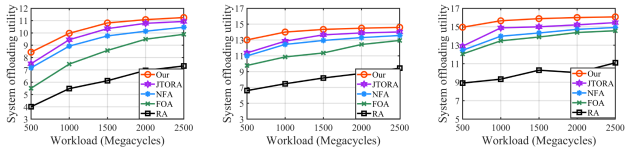
(a) low-resource (b) medium-resource (c) high-resource

Fig. 6. System offloading utility when varying MDs.



(a) low-resource (b) medium-resource (c) high-resource

Fig. 7. System offloading utility when varying task input data.



(a) low-resource (b) medium-resource (c) high-resource

Fig. 8. System offloading utility when varying task workloads.

B. Simulation Results

Below, we first present the system offloading utility and reliability achieved by our proposed scheme and four comparative approaches when changing MDs, task input data and task workloads, and then show how the users' preferences influence the performance of our scheme.

1) System Offloading Utility: Fig. 6 presents the system offloading utility obtained by using our proposed scheme and four benchmark approaches JTORA, NFA, FOA, RA when changing resource and MD settings. Clearly, our proposed scheme always outperforms the other four benchmark approaches in improving system offloading utility no matter which resource and MD settings are used. Comparing with the four benchmark approaches, our proposed scheme can improve system offloading utility by 6.1%, 8.1%, 23.8%, and 63.2% on average, respectively. The highest improvement is up to 84.0%, in the medium-resource scenario of Fig. 6(b) with $N = 20$ when compared to RA. The utility improvement realized by our proposed scheme mainly benefits from our utility-aware resource allocation algorithm which uses the convex optimization technique to allocate computing resources for each offloaded task to maximize offloading utility. In addition, from Fig. 6 we can also observe that the system offloading utility derived by our proposed scheme and the other four benchmark algorithms all increase along with the growth of the MU-MEC system's resource capacity. The reason is that more bandwidth and computing resources are provided by the MU-MEC system, more MD users can offload their tasks for obtaining higher system offloading utility.

To have a comprehensive evaluation, Figs. 7 and 8 show the system offloading utility obtained by using our scheme and the four benchmark approaches when changing the input data size and workload of tasks, respectively. Similar to Figs. 6, 7, and 8 also demonstrate that the system offloading utility realized by

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

TABLE V
SYSTEM RELIABILITY WHEN VARYING MDS

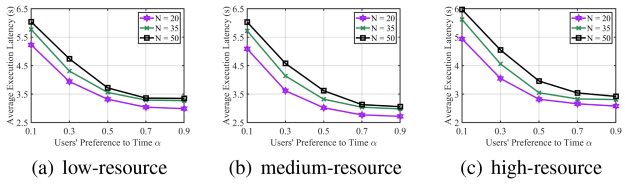
	low-resource			medium-resource			high-resource			Avg.
	$N =$			$N =$			$N =$			
	20	35	50	20	35	50	20	35	50	
NFA	0.949	0.927	0.847	0.964	0.912	0.870	0.974	0.932	0.889	0.918
JTORA	0.940	0.833	0.792	0.962	0.899	0.805	0.974	0.921	0.895	0.891
FOA	0.938	0.831	0.775	0.962	0.890	0.795	0.972	0.919	0.841	0.880
RA	0.913	0.822	0.720	0.938	0.840	0.786	0.951	0.861	0.822	0.850
Our	0.978	0.967	0.955	0.981	0.948	0.928	0.986	0.945	0.910	0.955

TABLE VI
SYSTEM RELIABILITY WHEN VARYING INPUT DATA

	low-resource			medium-resource			high-resource			Avg.
	$d_i =$			$d_i =$			$d_i =$			
	200	600	1000	200	600	1000	200	600	1000	
NFA	0.949	0.920	0.899	0.956	0.923	0.903	0.962	0.938	0.910	0.929
JTORA	0.942	0.916	0.894	0.955	0.920	0.897	0.960	0.932	0.908	0.925
FOA	0.933	0.908	0.884	0.935	0.911	0.890	0.938	0.915	0.896	0.912
RA	0.913	0.890	0.870	0.916	0.892	0.880	0.920	0.903	0.885	0.897
Our	0.968	0.939	0.924	0.979	0.948	0.933	0.983	0.961	0.939	0.953

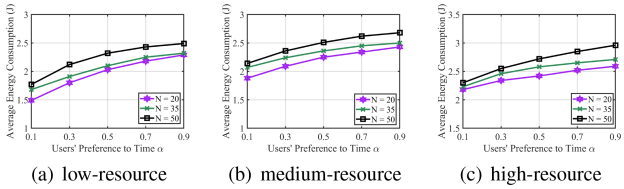
TABLE VII
SYSTEM RELIABILITY WHEN VARYING WORKLOAD

	low-resource			medium-resource			high-resource			Avg.
	$c_i =$			$c_i =$			$c_i =$			
	500	1500	2500	500	1500	2500	500	1500	2500	
NFA	0.950	0.942	0.935	0.962	0.955	0.948	0.966	0.961	0.955	0.953
JTORA	0.946	0.938	0.930	0.958	0.951	0.942	0.960	0.952	0.946	0.947
FOA	0.932	0.923	0.915	0.940	0.934	0.926	0.945	0.937	0.930	0.931
RA	0.910	0.901	0.893	0.922	0.916	0.908	0.930	0.923	0.919	0.914
Our	0.970	0.963	0.956	0.982	0.974	0.968	0.986	0.979	0.972	0.972



(a) low-resource (b) medium-resource (c) high-resource

Fig. 9. Average execution latency when varying α.



(a) low-resource (b) medium-resource (c) high-resource

Fig. 10. Average energy consumption when varying α.

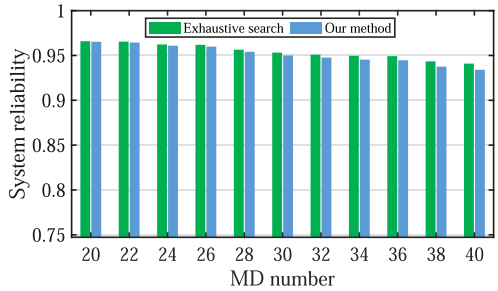


Fig. 11. Reliability achieved by our task assignment algorithm and the exhaustive search method.

our proposed scheme is always the highest among that of the five approaches regardless of task data size and workload. For example, in the high-resource scenario of Fig. 7(c) with data size $d_i = 800$ when compared to RA, our scheme achieves up to 104.6% increase in utility. From the two figures we also find that the system offloading utility decreases with the growth of task data size d_i, whereas it increases with the growth of task workload c_i. The observation indicates that tasks having smaller data size and larger workload are more suitable for offloading to execute on ESs remotely.

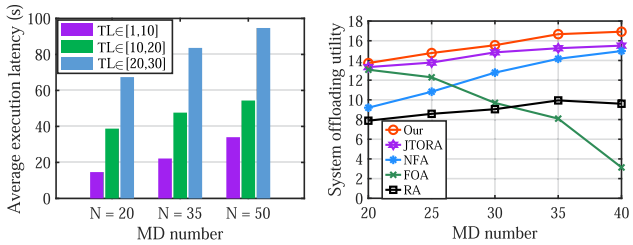
2) System Reliability: Tables V, VI, and VII list the system-level reliability obtained by using our proposed scheme and four benchmark approaches JTORA [13], NFA [44], FOA, RA when changing MDs, task input data and task workloads. As can be seen from the results, the average system reliability realized by our scheme over different settings is always the highest among the five approaches regardless of MDs, task input data and task workloads.

Table V show that given the resource setting, as the number of MDs increases, the system reliability achieved by the five approaches all exhibit a downtrend. However, unlike JTORA [13], NFA [44], FOA and RA that sometimes violate the reliability constraint (i.e., $\mathcal{R}_{\text{sys}} \geq \mathcal{R}_{\text{th}} = 0.9$), our proposed scheme always maintains a high-level system reliability and therefore meet the reliability constraint. Concretely, the average system reliability obtained by using our proposed scheme and the other four benchmark approaches are 0.955, 0.891, 0.918, 0.880, and 0.850, respectively. Particularly, in the low-resource scenario of Table V, comparing with the other four benchmark approaches, our scheme achieves up to 12.9%, 20.7%, 23.4% and 32.8% increase in system reliability, respectively. As presented in Tables VI and VII, the same observation can be found when changing task input data and task workloads. In the three tables, the average system reliability of our proposed scheme over different settings is 0.955, 0.953, and 0.973, respectively. They all satisfy the reliability constraint. Compared to the other four benchmark approaches, the highest system reliability obtained by using our scheme is owing to our task-to-ES assignment algorithm which assigns unreliable tasks to reliable ESs for improving system reliability.

Fig. 11 further compares the system reliability between our task-to-ES assignment algorithm and the exhaustive search method as we vary the number of MDs. The reliabilities are obtained by averaging the results of 100 repeated experiments. It can be observed that our algorithm achieves system reliability that closely matches the exhaustive search method across all numbers of MDs, with small differences averaging at 0.8%. This validates the efficacy of our proposed algorithm in enhancing reliability.

3) Effect of Users' Preferences: To fully validate our proposed scheme, Figs. 9 and 10 plot the average execution latency and energy consumption of all MD users when varying the users' preferences to latency (i.e., α_i) from 0.1 to 0.9. Since the users' preferences to latency and energy, represented by α_i and β_i, hold for $\alpha_i + \beta_i = 1$, when we explore the effect of users' preferences on latency, the effect of users' preferences on energy can be readily derived and it would be opposite to that of latency. This is the reason why we only give the results of average latency and energy consumption when changing α_i.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.



(a) Latency when varying MDs (b) Utility when varying MDs

Fig. 12. Experimental results on the testbed.

As clearly shown in Figs. 9 and 10, when the preference factor α_i increases, the average execution latency decreases at the expense of higher average energy consumption. This is reasonable since i) our scheme would tend to optimize execution latency when determining task offloading and computing resource allocation operations, according to the users' preferences, and ii) latency and energy are two conflicting goals which cannot be optimized simultaneously. Meanwhile, when the preference factor α_i increases to a certain level, the average execution latency and energy consumption tend to be stable. This is because the users' preferences have no impact on the offloading utility when the limited resources of ESs are exhausted. Besides, the figures also indicate that the users would experience higher average execution latency and larger energy dissipation when the system has more MD users. The reason is that, when more users compete for limited resources, a user is less likely to get benefits from offloading its task.

C. Testbed Platform Validation

Fig. 12 shows the experimental results on the testbed. It is crucial to acknowledge that variations in hardware configurations and task parameters lead to discrepancies between results obtained from testbed experiments and those from simulations. Fig. 12(a) demonstrates the influence of MD number N and task load TL on the average execution latency of tasks in our algorithm. With an escalating task load and a growing number of MDs participating in task offloading, the average execution latency tends to be prolonged. This is due to the limited ES and bandwidth resources, which cannot simultaneously cater to the offloading demands of all MDs. Our proposed scheme could mitigate this contradiction by evaluating the system's offloading utility and devising optimal offloading strategies. Fig. 12(b) showcases the system offloading utility obtained by using our proposed scheme and four benchmark approaches (i.e., JTORA, NFA, FOA, and RA) for varying numbers of MDs. Our scheme consistently outperforms benchmark approaches in terms of system offloading utility under varying MDs. The system unloading utility achieved by our scheme is higher than that of benchmark approaches by 43.1% on average. This is because our methodology integrates theoretical analysis with the innovative design of the efficient heuristic algorithm, enabling a significant enhancement in QoE. The results mentioned above validate the superior performance of the proposed methodology in real-world scenarios.

TABLE VIII
RESOURCE CONSUMED BY DIFFERENT ALGORITHMS

	Time consumption (s)				Energy consumption (J)			
	$N = 20$	$N = 35$	$N = 50$	Avg.	$N = 20$	$N = 35$	$N = 50$	Avg.
NFA	1.28914	2.43378	3.81235	2.51176	61.25282	113.71470	177.03457	117.33403
JTORA	0.01206	0.10597	0.38659	0.16821	0.72775	3.75776	13.00499	5.83017
FOA	0.00006	0.00007	0.00004	0.00006	0.00048	0.00050	0.00053	0.00050
RA	0.00031	0.00020	0.00025	0.00025	0.00340	0.00551	0.00797	0.00563
Our	0.04167	0.08514	0.16619	0.09767	1.87485	3.64462	7.22389	4.24779

TABLE IX
PERFORMANCE EVALUATION FOR DIFFERENT ALGORITHMS

(a) Weighted sum of energy consumption and latency

	$N =$			$d_i =$			$c_i =$			Avg.
	20	35	50	200	600	1000	500	1500	2500	
JTSP	0.14655	0.13042	0.13416	0.14488	0.20021	0.15351	0.11830	0.19276	0.13247	0.15036
ILRM	0.31059	0.15987	0.15228	0.25291	0.36682	0.20149	0.14274	0.30099	0.22663	0.23493
Our	0.08217	0.10593	0.12143	0.13406	0.14106	0.11380	0.07865	0.16516	0.12074	0.11811

(b) Transmission reliability

	$N =$			$d_i =$			$c_i =$			Avg.
	20	35	50	200	600	1000	500	1500	2500	
JTSP	0.98843	0.96929	0.94852	0.96929	0.96929	0.96929	0.96929	0.96929	0.96929	0.96911
Our	0.99999	0.99999	0.99999	0.99999	0.99999	0.99999	0.99999	0.99999	0.99999	0.99999

(c) Execution reliability

	$N =$			$d_i =$			$c_i =$			Avg.
	20	35	50	200	600	1000	500	1500	2500	
ILRM	0.90004	0.90448	0.88563	0.90086	0.90320	0.90452	0.94034	0.90876	0.87589	0.90264
Our	0.97822	0.95336	0.93376	0.95619	0.95672	0.95414	0.98744	0.96087	0.93451	0.95724

D. Resource Consumption of Algorithm

Table VIII compares the resource consumption (including time and energy consumption) of executing our scheme and four benchmark algorithms on the base station. From the results in the table, we can derive the following two observations. 1) An increase in the number of MD leads to higher time and energy consumption for NFA, JTORA, and our scheme. This increase occurs because the computational demands per iteration rise as the number of MDs grows. In contrast, the FOA and RA algorithms make offloading decisions directly, without relying on iterative processes, thus their time and energy consumption are less impacted by the number of devices. 2) Our scheme significantly reduces both time and energy consumption when compared to JTORA and NFA, achieving average reductions of 41.9% and 96.1% in time, and 27.1% and 96.4% in energy, respectively. It is worth noting that our scheme only demonstrates higher time and energy consumption compared to JTORA when $N = 20$; in all other scenarios, ours are lower. This is mainly due to the fact that an increase in MDs affects the number of iterations required by JTORA more than it does for our scheme. Additionally, NFA consistently demands much more time and energy than the others because of its complex computational nature.

E. Comparison of Transmission and Execution Performance

Table IX compares the proposed scheme with the JTSP [45] method, which focuses on optimizing communication, and the ILRM [46] method, which focuses on optimizing computation. This comparison considers changes in the number of MDs, task input data, and task workloads. Table IX(a) lists the weighted sum of normalized latency and energy consumption (namely

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

task execution overhead) of three algorithms. Our method consistently has lower overhead than JTSP and ILRM across different MDs, task input data, and task workloads, achieving an average reduction of 21.4% and 49.7%, respectively. Table IX(b) presents the transmission reliability achieved by our scheme and JTSP. Our algorithm shows outstanding performance, maintaining a reliability level of 0.99999 across all tested scenarios. In contrast, JTSP has an average transmission reliability of 0.96911. This high-level reliability of our scheme ensures more stable data transmission, effectively reducing the risk of data errors during communication. Table IX shows that our scheme also outperforms ILRM in execution reliability. With an average execution reliability of 0.95724, compared to 0.90264 achieved by ILRM, our scheme demonstrates a greater ability to accurately manage tasks under varying loads.

IX. Summary and Future Work

This paper focused on a task offloading and scheduling problem aiming to optimize the QoE of MU-MEC systems while maintaining a certain level of reliability. To solve the problem, we designed a task-to-ES assignment heuristic which improves system reliability on the basis of a reliability-optimality analysis, a resource allocation heuristic that uses the convex optimization technique to allocate computing resources to offloaded tasks for improving system QoE, and a BPSO based task offloading heuristic that searches the best task offloading strategy with the global optimum QoE and satisfactory system reliability. Simulation and testbed experiment results verify the efficacy of our proposed scheme.

This paper adopts the exponential distribution to model fault rate and reliability. Based on this exponential model, we derive a theorem concerning the reliability-optimality of task-to-ES assignment. However, to improve the scalability of our theorem and algorithm, it is important to consider other reliability models, such as the Weibull distribution. In addition, we use a simplified task model that assumes each device has one computation task to execute during each time slot. However, practical applications may be more complex. To broaden the applicability of our scheme, we need to cope with the dynamics associated with task arrivals. Therefore, in the near future, we plan to develop new task offloading and scheduling algorithms for more complicated MEC systems.

References

[1] W. Jiang, J. Zhou, P. Cong, G. Zhang, and S. Hu, "QoE and reliability-aware task scheduling for multi-user mobile-edge computing," in *Wireless Algorithms, Systems, and Applications*. Berlin, Germany: Springer, 2022, pp. 380–392.

[2] G. Bhat, K. Bagewadi, H. G. Lee, and U. Y. Ogras, "REAP: Runtime energy-accuracy optimization for energy harvesting IoT devices," in *Proc. 56th ACM/IEEE Des. Automat. Conf.*, 2019, pp. 1–6.

[3] J. Zhou et al., "Security-critical energy-aware task scheduling for heterogeneous real-time MPSoCs in IoT," *IEEE Trans. Services Comput.*, vol. 13, no. 4, pp. 745–758, Jul./Aug. 2020.

[4] J. Li et al., "Maximizing user service satisfaction for delay-sensitive IoT applications in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 5, pp. 1199–1212, May 2022.

[5] "Internet of Things: Key stats for 2022," 2022. [Online]. Available: https://techinformed.com/internet-of-things-key-stats-for-2022/

[6] H. Yuan and M. Zhou, "Profit-maximized collaborative computation offloading and resource allocation in distributed cloud and edge computing systems," *IEEE Trans. Automat. Sci. Eng.*, vol. 18, no. 3, pp. 1277–1287, Jul. 2021.

[7] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, "On multi-access edge computing: A survey of the emerging 5 G network edge cloud architecture and orchestration," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 3, pp. 1657–1681, Third Quarter 2017.

[8] T. G. Rodrigues, K. Suto, H. Nishiyama, N. Kato, and K. Temma, "Cloudlets activation scheme for scalable mobile edge computing with transmission power control and virtual machine migration," *IEEE Trans. Comput.*, vol. 67, no. 9, pp. 1287–1300, Sep. 2018.

[9] Z. He, K. Li, and K. Li, "Cost-efficient server configuration and placement for mobile edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 9, pp. 2198–2212, Sep. 2022.

[10] J. Wang, J. Hu, G. Min, W. Zhan, A. Y. Zomaya, and N. Georgalas, "Dependent task offloading for edge computing based on deep reinforcement learning," *IEEE Trans. Comput.*, vol. 71, no. 10, pp. 2449–2461, Oct. 2022.

[11] Y. Zhan, S. Guo, P. Li, and J. Zhang, "A deep reinforcement learning based offloading game in edge computing," *IEEE Trans. Comput.*, vol. 69, no. 6, pp. 883–893, Jun. 2020.

[12] M. Du, Y. Wang, K. Ye, and C. Xu, "Algorithmics of cost-driven computation offloading in the edge-cloud environment," *IEEE Trans. Comput.*, vol. 69, no. 10, pp. 1519–1532, Oct. 2020.

[13] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.

[14] Z. Hu, F. Zeng, Z. Xiao, B. Fu, H. Jiang, and H. Chen, "Computation efficiency maximization and QoE-provisioning in UAV-enabled MEC communication systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1630–1645, Second Quarter 2021.

[15] P. Cong, J. Zhou, W. Jiang, M. Chen, S. Hu, and K. Li, "Improving reliability and sustainability of hazard-aware cyber-physical systems," *IEEE Trans. Sustain. Comput.*, vol. 9, no. 3, pp. 271–282, May/Jun. 2024.

[16] Z. Kuang, L. Li, J. Gao, L. Zhao, and A. Liu, "Partial offloading scheduling and power allocation for mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 4, pp. 6774–6785, Aug. 2019.

[17] J. Wang, J. Hu, G. Min, A. Y. Zomaya, and N. Georgalas, "Fast adaptive task offloading in edge computing based on meta reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 1, pp. 242–253, Jan. 2021.

[18] Z. Chang, L. Liu, X. Guo, and Q. Sheng, "Dynamic resource allocation and computation offloading for IoT fog computing system," *IEEE Trans. Ind. Inform.*, vol. 17, no. 5, pp. 3348–3357, May 2021.

[19] S. Gupta and J. Chakareski, "Lifetime maximization in mobile edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3310–3321, Mar. 2020.

[20] M. Guo, W. Wang, X. Huang, Y. Chen, L. Zhang, and L. Chen, "Lyapunov-based partial computation offloading for multiple mobile devices enabled by harvested energy in MEC," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 9025–9035, Jun. 2022.

[21] L. Dong, W. Wu, Q. Guo, M. N. Satpute, T. Znati, and D. Z. Du, "Reliability-aware offloading and allocation in multilevel edge computing system," *IEEE Trans. Rel.*, vol. 70, no. 1, pp. 200–211, Mar. 2021.

[22] K. Peng, B. Zhao, M. Bilal, and X. Xu, "Reliability-aware computation offloading for delay-sensitive applications in mec-enabled aerial computing," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 3, pp. 1511–1519, Sep. 2022.

[23] W. Liang, Y. Ma, W. Xu, Z. Xu, X. Jia, and W. Zhou, "Request reliability augmentation with service function chain requirements in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 12, pp. 4541–4554, Dec. 2022.

[24] J. Mei, Z. Tong, K. Li, L. Zhang, and K. Li, "Energy-efficient heuristic computation offloading with delay constraints in mobile edge computing," *IEEE Trans. Services Comput.*, vol. 16, no. 6, pp. 4404–4417, Nov./Dec. 2023.

[25] S. Chu, C. Gao, M. Xu, K. Ye, Z. Xiao, and C. Xu, "Efficient multi-task computation offloading game for mobile edge computing," *IEEE Trans. Services Comput.*, vol. 17, no. 1, pp. 30–46, Jan./Feb. 2024.

[26] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Cooperative task offloading and block mining in blockchain-based edge computing with multi-agent deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2021–2037, Apr. 2023.

[27] M. Salehi et al., "Drvs: Power-efficient reliability management through dynamic redundancy and voltage scaling under variations," in *Proc. IEEE/ACM Int. Symp. Low Power Electron. Des.*, 2015, pp. 225–230.

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.

[28] Z. Zhou, P. Liu, J. Feng, Y. Zhang, S. Mumtaz, and J. Rodriguez, "Computation resource allocation and task assignment optimization in vehicular fog computing: A contract-matching approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3113–3125, Apr. 2019.
[29] U. Saleem, Y. Liu, S. Jangsher, Y. Li, and T. Jiang, "Mobility-aware joint task scheduling and resource allocation for cooperative mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 360–374, Jan. 2021.
[30] L. Yin, J. Sun, J. Zhou, Z. Gu, and K. Li, "ECFA: An efficient convergent firefly algorithm for solving task scheduling problems in cloud-edge computing," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3280–3293, Sep./Oct. 2023.
[31] Y.-J. Choi, S. Park, and S. Bahk, "Multichannel random access in OFDMA wireless networks," *IEEE J. Sel. Areas Commun.*, vol. 24, no. 3, pp. 603–613, Mar. 2006.
[32] Z. Liao et al., "Distributed probabilistic offloading in edge computing for 6G-enabled massive Internet of Things," *IEEE Internet Things J.*, vol. 8, no. 7, pp. 5298–5308, Apr. 2021.
[33] L. Li et al., "Feedback control of real-time ethercat networks for reliability enhancement in CPS," in *Des. Automat. Test Eur. Conf. Exhibit.*, 2018, pp. 688–693.
[34] J. Zhou, X. S. Hu, Y. Ma, J. Sun, T. Wei, and S. Hu, "Improving availability of multicore real-time systems suffering both permanent and transient faults," *IEEE Trans. Comput.*, vol. 68, no. 12, pp. 1785–1801, Dec. 2019.
[35] J. Zhou, K. Cao, X. Zhou, M. Chen, T. Wei, and S. Hu, "Throughput-conscious energy allocation and reliability-aware task assignment for renewable powered in-situ server systems," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 41, no. 3, pp. 516–529, Mar. 2022.
[36] D. Zhu, R. Melhem, and D. Mosse, "The effects of energy management on reliability in real-time embedded systems," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Des.*, 2004, pp. 35–40.
[37] M. A. Haque, H. Aydin, and D. Zhu, "On reliability management of energy-aware real-time systems through task replication," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 3, pp. 813–825, Mar. 2017.
[38] Z. Beheshti, "A time-varying mirrored s-shaped transfer function for binary particle swarm optimization," *Inf. Sci.*, vol. 512, pp. 1503–1542, 2020.
[39] Z. Zhao et al., "A novel framework of three-hierarchical offloading optimization for MEC in industrial IoT networks," *IEEE Trans. Ind. Inform.*, vol. 16, no. 8, pp. 5424–5434, Aug. 2020.
[40] S. Mirjalili and A. Lewis, "S-shaped versus v-shaped transfer functions for binary particle swarm optimization," *Swarm Evol. Computation*, vol. 9, pp. 1–14, 2013.
[41] J. Kennedy and R. Eberhart, "A discrete binary version of the particle swarm algorithm," in *Proc. IEEE Int. Conf. Syst. Man Cybern. Comput. Cybernet. Simul.*, 1997, pp. 4104–4108 vol.5.
[42] Nvidia, (n.d.). Accessed: Jun. 15, 2024. [Online]. Available: https://www.nvidia.com/en-us/autonomous-machines/embedded-systems/
[43] Yolov5, 2020. [Online]. Available: https://github.com/ultralytics/yolov5
[44] Y. Zhang, J. Zhou, L. Sun, J. Mao, and J. Sun, "A novel firefly algorithm for scheduling bag-of-tasks applications under budget constraints on hybrid clouds," *IEEE Access*, vol. 7, pp. 151888–151901, 2019.
[45] C. Dong, Y. Tian, Z. Zhou, W. Wen, and X. Chen, "Joint power allocation and task offloading for reliability-aware services in noma-enabled MEC," *IEEE Trans. Wireless Commun.*, vol. 23, no. 7, pp. 7537–7551, Jul. 2024.
[46] X. Hou et al., "ILRM: Imitation learning based resource management for integrated CPU-GPU edge systems with renewable energy sources," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, to be published, doi: 10.1109/TCAD.2024.3513892.



Junlong Zhou (Member, IEEE) is an associate professor with the School of Computer Science and Engineering, Nanjing University of Science and Technology, China. His research interests include edge computing, cloud computing, service computing and embedded systems, where he has published 120 papers, including more than 40 in premier IEEE/ACM Transactions. He has been a subject area editor for the *Journal of Systems Architecture* and an associate editor for the *Journal of Circuits, Systems, and Computers* and the IET Cyber-Physical Systems: Theory & Applications. He received the Best Paper Awards from IEEE iThings 2020, IEEE CPSCom 2022, and IEEE ICITES 2024.



Xiangpeng Hou (Student Member, IEEE) received the BS degree from the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, China, in 2021, where he is currently working toward the PhD degree. His research interest includes edge intelligence.



Yue Zeng received the PhD degree from Nanjing University, China, in 2023. He is currently an associate professor with the Nanjing University of Science and Technology. He has published more than a dozen papers in top journals and conferences, including *IEEE Transactions on Computers*, *IEEE Transactions on Services Computing*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Communications*, *Transactions on Cloud Computing* and IEEE CVPR. His research interests include edge intelligence, federated learning, and serverless computing.



Peijin Cong is an associate professor with the School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, China. Her research interests include the area of cloud computing and service computing, where she has published 30 refereed papers, more than half of which appeared in IEEE/ACM Transaction papers.



Weiming Jiang received the MS degree in computer science from the Nanjing University of Science and Technology, China, in 2023. His research interests include cloud computing and edge computing.



Song Guo (Fellow, IEEE) is a chair professor with the Department of Computer Science and Engineering, Hong Kong University of Science and Technology. He also holds a Changjiang chair professorship awarded by the Ministry of Education of China. His research interests include edge AI, mobile computing, and distributed systems. He has been recognized as a highly cited Researcher (Web of Science) and was the recipient of more than 14 best paper awards from IEEE/ACM conferences, journals, and technical committees. He is the Editor-in-Chief of IEEE Transactions on Cloud Computing and the founding editor-in-chief of *IEEE Open Journal of the Computer Society*. He has been named on the editorial board of a number of prestigious international journals like *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Emerging Topics in Computing*, *IEEE Transactions on Sustainable Computing*, *ACM Computing Surveys*, etc. He is a fellow of the Canadian Academy of Engineering (FCAE), a Member of Academia Europaea (MAE), a distinguished member of the ACM, and a fellow of the Asia-Pacific Artificial Intelligence Association (FAAIA).

Authorized licensed use limited to: University of Pittsburgh. Downloaded on July 07,2026 at 13:30:26 UTC from IEEE Xplore. Restrictions apply.